Collaborative Edge Computing With FPGA-Based CNN Accelerators for Energy-Efficient and Time-Aware Face Tracking System

Xing Liu, Jianfeng Yang, Chengming Zou, Qimei Chen, *Member, IEEE*, Xin Yan, Yuao Chen, and Chenran Cai

***Abstract*— Convolutional neural networks (CNNs) have become the critical technology to realize face detection and face recognition in the face tracking (FT) system. However, traditional CNNs usually have nontrivial computational time and high energy consumption, making them inappropriate to be deployed in the large-scale time-sensitive FT system. To address this challenge, we design an artificial intelligence and Internet of Things (AIoT) empowered edge-cloud collaborative computing (ECCC) system based on the energy-efficient field-programmable gate array (FPGA)-based CNN accelerators for the purpose of realizing a low-latency and low-power FT. First, we present the AIoT-empowered ECCC system architecture, which consists of an intelligent computing subsystem, an Internet-of-Things (IoT) subsystem, an edge-cloud collaborative subsystem, and an application subsystem. In what follows, we investigate the enabling technologies for these subsystems. Thereafter, we develop an FPGA-based hardware accelerator dedicated to the compact MobileNet CNN by using the hardware design techniques, such as systolic array, matrix tiling, fixed-point precision, and parallelism. Furthermore, we integrate the FPGA accelerators with CPUs and GPUs to build a context-aware CPU/GPU/FPGA heterogeneous computing system. Finally, we implement a delay-aware energy-efficient scheduling algorithm dedicated to this heterogeneous system. With the above hardware and software codesign mechanism, the energy cost and execution time of CNNs can be decreased significantly. The real-world experiments on the CPU/GPU/FPGA-based ECCC system proved the effectiveness of the proposed schemes in reducing the latency and improving the power efficiency of the FT system.**

Manuscript received October 30, 2020; revised February 3, 2021; accepted February 10, 2021. Date of publication February 25, 2021; date of current version January 31, 2022. This work was supported in part by the Natural Science Foundation of Xinjiang Province of China under Grant 2020D01A130, in part by the National Natural Science Foundation of China under Grant 61702387 and Grant 61771354, in part by the Hubei Provincial Key Research and Development Program under Grant 2020BAA001, and in part by the National Innovation and Entrepreneurship Training Program for College Students under Grant 202010497040. *(Corresponding author: Jianfeng Yang.)*

Xing Liu is with the Hubei Key Laboratory of Transport Internet of Things, Wuhan University of Technology, Wuhan 430070, China, and also with the Western Research Institute of Yangtze University, China (e-mail: liu.xing@whut.edu.cn).

Jianfeng Yang, Qimei Chen, and Yuao Chen are with the School of Electronic Information, Wuhan University, Wuhan 430072, China (e-mail: yjf@whu.edu.cn; chenqimei@whu.edu.cn; chenyuao@whu.edu.cn).

Chengming Zou is with the Hubei Key Laboratory of Transport Internet of Things, Wuhan University of Technology, Wuhan 430070, China (e-mail: zoucm@whut.edu.cn).

Xin Yan and Chenran Cai are with the School of Computer Science and Technology, Wuhan University of Technology, Wuhan 430070, China (e-mail: yanxin@whut.edu.cn; crcai@whut.edu.cn).

Digital Object Identifier 10.1109/TCSS.2021.3059318

***Index Terms*— Accelerator, collaborative edge computing, convolutional neural network (CNN), face tracking (FT).**

I. Introduction

With the recent advances in cameras, signal processing, and web technologies, face tracking (FT) has become an important research direction and has played a crucial role in many surveillance scenarios, such as crime tracking, human emotion detection, and human–machine interaction [1]. Taking crime tracking as an example, the identities of the criminals can be obtained by performing face recognition to the facial images captured by cameras, and then, the criminals can be tracked by performing data fusion to the face recognition information from different cameras.

FT is a design work that involves several computer vision techniques, such as face detection, face recognition, and motion tracking. Since convolutional neural networks (CNNs) have strong ability in learning nonlinear features and show high recognition accuracy, they have been widely applied to realize face detection and face recognition in the FT system. Yang *et al.* [2] and Mahbub *et al.* [3] used CNNs to handle severe face detection occlusion problems. Ranjan *et al.* [4] applied CNNs to achieve simultaneous face detection, facial landmarks localization, head pose estimation, and gender recognition. Deng *et al.* [5] utilized CNNs to address the challenges of large modality discrepancy and insufficient training samples in heterogeneous face recognition (HFR) system. Ding *et al.* [6] run CNNs to overcome the challenges of severe image blur, dramatic pose variations, and occlusion in video-based face recognition. Wang and Guo [7] applied the CNN technology to learn local and multiscale face representations together. Large amounts of related research work have demonstrated the significance of using CNN technology to perform face recognition and detection in the FT system.

However, there exist several challenges in deploying CNNs to the FT system on a large scale. First, the state-of-the-art CNN models are trending toward more layers and more complicated structures to achieve higher accuracy, which makes the standard CNNs become computation-intensive and memory expensive [8]. Specifically, most CNNs require over a billion operations for each input image and need hundreds of megabyte memory for the parameters. The high computational complexity and large memory footprint require these CNNs to be offloaded to the resource-abundant cloud servers. However,

2329-924X © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

this offloading suffers from network congestion and service latency [9]–[12]. Second, there are usually large amounts of facial images to be detected and recognized in the large-scale FT system, which requires each image to be processed quickly within a short time. However, most CNNs have non-trivial execution time due to the high computing complexity. Consequently, the computing acceleration becomes critical for CNNs. Third, there are many FT cases in which a large number of surveillance devices and edge servers need to be deployed, e.g., the urban FT surveillance system is usually equipped with large amounts of surveillance devices. In these cases, the FT system can be regarded as a data center to a certain degree, and it can cause huge energy cost [13]. Therefore, the energy optimization of the FT system to enable it economically deployed in the actual industry is another challenging problem.

To address the above challenges, the work in this article aims to build an energy-efficient and time-aware FT system by performing edge-cloud collaborative computing (ECCC) based on the field-programmable gate array (FPGA)-based MobileNet CNN accelerators. First, an ECCC system is proposed, which classifies the facial images in terms of the facial quality and sends the low-quality facial images, e.g., the images with high crowd density and varying illuminations, to the remote cloud servers, while keeping the other images to be processed on the local servers. In this way, the advantages of both cloud computing's high performance and edge computing's low latency can be integrated. Second, the compact MobileNet CNN model [14], [15], which optimizes the multiply–accumulate (MAC) operations and model parameters of CNNs while maintaining high accuracy by using depthwise separable convolution (DSC) [16], is used to substitute the standard CNN models in the FT system so that the computing time and energy cost of CNN computation in the FT system can be decreased significantly. Next, considering that FPGA is hardware programmable and can take advantage of the intrinsic parallel computing characteristics of CNNs, an FPGA-based hardware accelerator dedicated to MobileNet CNN is designed. By running MobileNet CNN on this hardware accelerator, the executing speed of CNNs can be improved, while the energy cost of CNNs can also be decreased. Furthermore, considering that CPUs show advantages in performing general-purpose tasks and GPUs have stronger performance when they have a similar logic density as FPGAs, the FPGA-based CNN accelerators are further combined with CPUs and GPUs to build a heterogeneous CPU/GPU/FPGA computing system. This integration enables the hardware architecture to become highly aware of the application contexts. Consequently, high computing performance can be achieved with lower energy cost. Finally, dedicated to the proposed CPU/GPU/FPGA heterogeneous computing architecture, a time-aware and energy-efficient task scheduling mechanism is studied to assign each task to the most appropriate computing core. With the above software and hardware codesign strategy, the FT tasks can be executed more efficiently with less time and low power.

The contributions of this article are as follows.

1) An artificial intelligence and Internet of Things (AIoT) empowered system architecture dedicated to the ECCC system is proposed, and the enabling technologies in different subsystems of this architecture are investigated. This AIoT-empowered ECCC system can promote the development of many emerging applications, such as fast face recognition, autonomous driving, and big data processing acceleration.
2) An FPGA-based MobileNet CNN accelerator is designed, and a heterogeneous CPU/GPU/FPGA computing architecture based on this accelerator is built to make the CNN-based applications run to completion with lower energy cost and less execution time.
3) An energy-efficient and time-aware task scheduling algorithm dedicated to the above heterogeneous system is studied.
4) A real-world CPU/GPU/FPGA-based ECCC system is implemented to evaluate the performance of the proposed schemes.

The organization of this article is as follows. In Section II, the related work on using CNNs to perform face detection and face recognition is introduced. The related work on designing FPGA-based CNN accelerators is also presented. In Section III, the system architecture of the AIoT-empowered ECCC system is designed, and the enabling technologies of this architecture are also discussed. In Section IV, the design of an FPGA-based MobileNet CNN accelerator is investigated. In Section V, the delay-aware and energy-efficient scheduling algorithm for the heterogeneous CPU/GPU/FPGA system is explored. In Section VI, the experiments are conducted to evaluate the energy cost and executed time of CNNs in the FT system. Finally in Section VII, the conclusion is given.

II. Related Work

A. Face Detection and Recognition With CNNs

Face detection and face recognition are both important parts of FT, and they have been popularly implemented by using CNN technologies.

Many studies in recent years have applied CNNs to perform face recognition. Yang *et al.* [2] proposed a deep CNN for face detection by using facial attribute-based supervision, and the result shows that this way can effectively enhance the capability of a face detection network in handling severe occlusions. Mahbub *et al.* [3] presented a two-step deep CNN-based method to perform attribute detection in partially occluded faces by considering the relative strength of different facial segments in detecting different facial attributes. Ranjan *et al.* [4] proposed a deep multitask learning framework based on CNNs for simultaneous face detection, facial landmarks localization, head pose estimation, and gender recognition.

CNNs have also been widely applied to realize face recognition. Yin and Liu [17] proposed a multitask CNN for face recognition by using the identity classification as the main task and the other features, such as pose, illumination, and expression as the side tasks. Deng *et al.* [5] aimed to develop an HFR system by identifying a person from different facial modalities such as visible and near-infrared images, and they have proposed a mutual component CNN to address the challenges of

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

large modality discrepancy and insufficient training samples in the HFR system. Ding and Tao [6] proposed a comprehensive framework based on CNNs to overcome the challenges of severe image blur, dramatic pose variations, and occlusion in video-based face recognition. They train the CNNs to learn blur-insensitive features automatically and design a trunk-branch ensemble CNN model to extract complementary information from holistic face images and patches cropped around facial components. Wang and Guo [7] proposed a new model called local and multiscale CNN (LS-CNN) to learn local and multiscale face representations together. They propose the harmonious multiscale network (HSNet) to learn multiscale features from different perspectives and then combine HSNet with spatial attention and channel attention to characterize informative regions and highlight important channels.

Since MobileNet CNNs use DS convolution and have less computing complexity and small memory cost, they have been popularly used to substitute the standard CNNs in many deep learning algorithms so that the execution speed and energy cost of CNNs can be decreased significantly. Sandler *et al.* [15] proposed the SSDLite to replace all the regular convolutions in the prediction layers of single-short detector (SSD) algorithm with separable convolutions. The experiment results show that SSDLite can dramatically reduce the parameter size of SSD from $14.8M$ to $2.1M$ and decrease the multiply–adds operations of SSD from $1.25B$ to $0.35B$. Chua *et al.* [18] developed a modern convolutional object detector based on SSD Mobilenet v1 to avoid the children dying from vehicular heatstroke. Xu [19] demonstrated that SSD with Mobilenet is incredibly fast and still accurate enough to implement a practical video surveillance system for commercial uses. Cai *et al.* [20] used MobileNet as the backbone network to perform feature extraction in the original YOLOv3 framework and applied this framework for fish detection in real breeding farm. The results show that the performance of the proposed fish detection model is improved obviously in this way.

As CNNs are the key technique to implement FT and MobileNet CNNs are usually used in deep learning algorithms to substitute the standard CNNs, the following sections focus on investigating how to design and implement a low-power and low-latency MobileNet CNN accelerator to empower the development of an energy-efficient and time-sensitive FT system.

B. FPGA-Based CNN Accelerators

Due to the low power and high flexibility, extensive studies have investigated the ways to design the energy-efficient CNN accelerators based on reprogrammable FPGAs, and a set of architecture design techniques for the FPGA accelerators have been studied.

Parallel computing and pipeline scheduling are the most commonly used hardware accelerating methods for designing FPGA accelerators and have been widely applied by many CNN accelerators [21]–[23].

Memory system optimization is also critical for the accelerator design and has drawn widespread research attention. Du *et al.* [21] optimized the energy efficiency of FPGA accelerators by avoiding unnecessary data movement. Chen *et al.* [24] decreased the expensive data movements significantly by reusing the data locally with a proposed spatial dataflow architecture.

Computing complexity optimization is another efficient approach to improve CNN accelerating performance. Lei *et al.* [22] accelerated CNNs by performing data quantization. Aimar *et al.* [25] accelerated the computing speed and reduce the memory footprint of CNNs by exploiting the sparsity of neuron activations in CNNs. Lu *et al.* [26] improved the performance of FPGA accelerators by compressing synapse weights in large-scale sparse neural networks. Xiao *et al.* [27] improved the FPGA accelerating efficiency by optimizing the conventional convolution algorithm with Winograd's minimal filtering theory.

Although there exist many CNN accelerating studies, they mostly focus on accelerating the standard CNNs on the resource-abundant cloud servers, and there is currently less attention paid to the CNN accelerators on the resource-constrained edge devices. In addition, there are a few studies that integrate FPGA-based CNN accelerators with CPUs and GPUs to make the hardware adapt to different application contexts. Therefore, the following sections focus on designing a low-power, low-latency CNN accelerator for the resource-constrained edge devices based on the CPU/GPU/FPGA heterogeneous computing architecture.

III. System Architecture

This section first presents an overview of the ECCC-based FT system and then proposes the architecture of the ECCC system. Later, the enabling technologies in different subsystems of this ECCC architecture are discussed.

A. Overview to FT System

The overview of the ECCC-based FT system is shown in Fig. 1. In this section, we take the crime tracking as an example to explain how this FT system works.

A set of cameras are deployed in different areas. These cameras capture the facial images and send them to the edge servers. Edge servers recognize these images based on CNNs and then identify the crimes by matching the extracted facial representations with the crimes' facial representations.

Since MobileNet CNN has less computing complexity and smaller parameter size, it is used to substitute the standard CNNs in the FT system. In this way, the latency and energy cost of FT system can be reduced. However, MobileNet CNN is incompetent to process the images with complex backgrounds, e.g., the images with high crowd density or varying illuminations. These images need to be processed by the more complicated CNN models with deeper network structure.

Therefore, the ECCC mechanism is implemented, which uses a cloud-edge task scheduler to classify the facial images in terms of face quality and dispatches the low-quality facial images to the remote clouds while keeping the high-quality ones run on the local edge servers. By this collaborative computing way, the advantages of both cloud computing's high

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

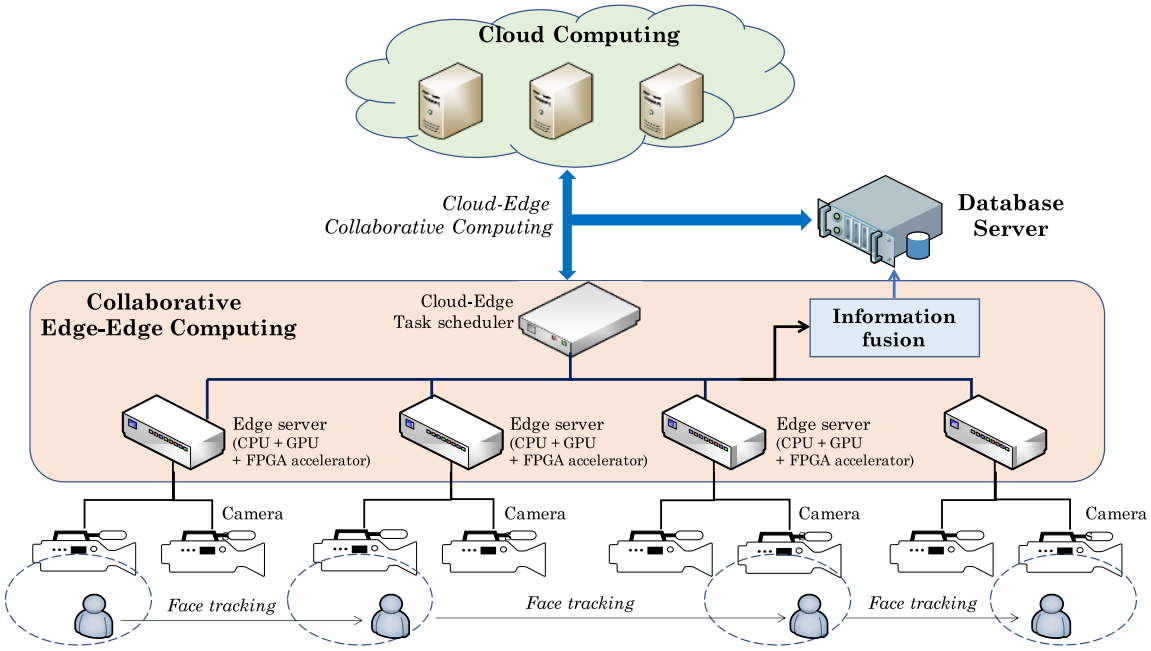

Fig. 1. Overview to the FT system.

computing performance and edge computing's low latency can be integrated.

Not only has the collaborative computing mechanism been applied between clouds and edges but also it has been used between edges and edges. As it is shown in Fig. 1, when the criminals move through different areas, they will be captured by different cameras and then identified by edge servers through face recognition. Next, the face recognition information of different edge servers will be shared and data fusion to this information will be performed. In this way, continuous tracking to the criminals in different areas can be realized.

B. Architecture of ECCC System

The system architecture of the ECCC system is shown in Fig. 2. It is empowered by the AIoT technology and it consists of four subsystems: intelligent computing subsystem, Internet-of-Things (IoT) subsystem, edge-cloud collaborative subsystem, and application subsystem.

1) Heterogeneous Computing Hardware Layer: CPUs are general-purpose processors and cannot demonstrate high computing performance when executing computation-intensive tasks. GPUs have stronger computing power than CPUs due to the higher computing parallelism. However, their inherent high energy consumption limits their deployment in large-scale computing centers. Since FPGAs show advantages in low energy consumption, low latency, and high reconfigurability, much recent work has investigated the ways to accelerate the computation-intensive tasks by FPGA accelerators [28].

However, different hardware architectures have different advantages [29], [30]. Although FPGA-based accelerators have many advantages, they cannot completely substitute CPUs and GPUs. Compared with FPGAs, CPUs are more appropriate to execute general-purpose tasks, and GPUs can achieve better computing performance when they have a similar logic density as FPGAs. Therefore, a superior way to build the computing architecture is to combine FPGA-based CNN accelerators with CPUs and GPUs, that is, the heterogeneous CPU/GPU/FPGA computing architecture, so that the hardware can become highly aware of the software run-time contexts and achieves high computing performance while keeping low energy cost.

2) Hardware Resource Management Layer: Dedicated to the heterogeneous CPU/GPU/FPGA computing platform, a task scheduling algorithm needs to be studied to assign each upper level task to the most appropriate underlying computing core to minimize the energy cost while guaranteeing the deadline constraint. The heterogeneous scheduling algorithms are usually classified into two kinds: heuristic ones and meta-heuristic ones. Also, to strengthen the scheduling performance in convergence speed, makespan, and energy efficiency, some work tries to combine two or more of the above algorithms, named hybrid scheduling algorithms.

The dynamic voltage and frequency scaling (DVFS) techniques are mostly combined with task scheduling algorithms to enhance the scheduling performance. DVFS can reduce the dynamic power consumption by adjusting the voltage and frequency of a computing unit dynamically in terms of the run-time contexts. It can strike the tradeoff between energy cost and execution time, which makes it suitable to solve the scheduling problems regarding joint optimization to both execution time and energy cost.

Computing resource virtualization is also significant for edge computing system in improving the hardware utilization efficiency, increasing computing scalability, as well as strengthening computing security. Docker is a set of platform-as-a-service (PaaS) products that use operating system (OS)-level virtualization technique to deliver software in

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

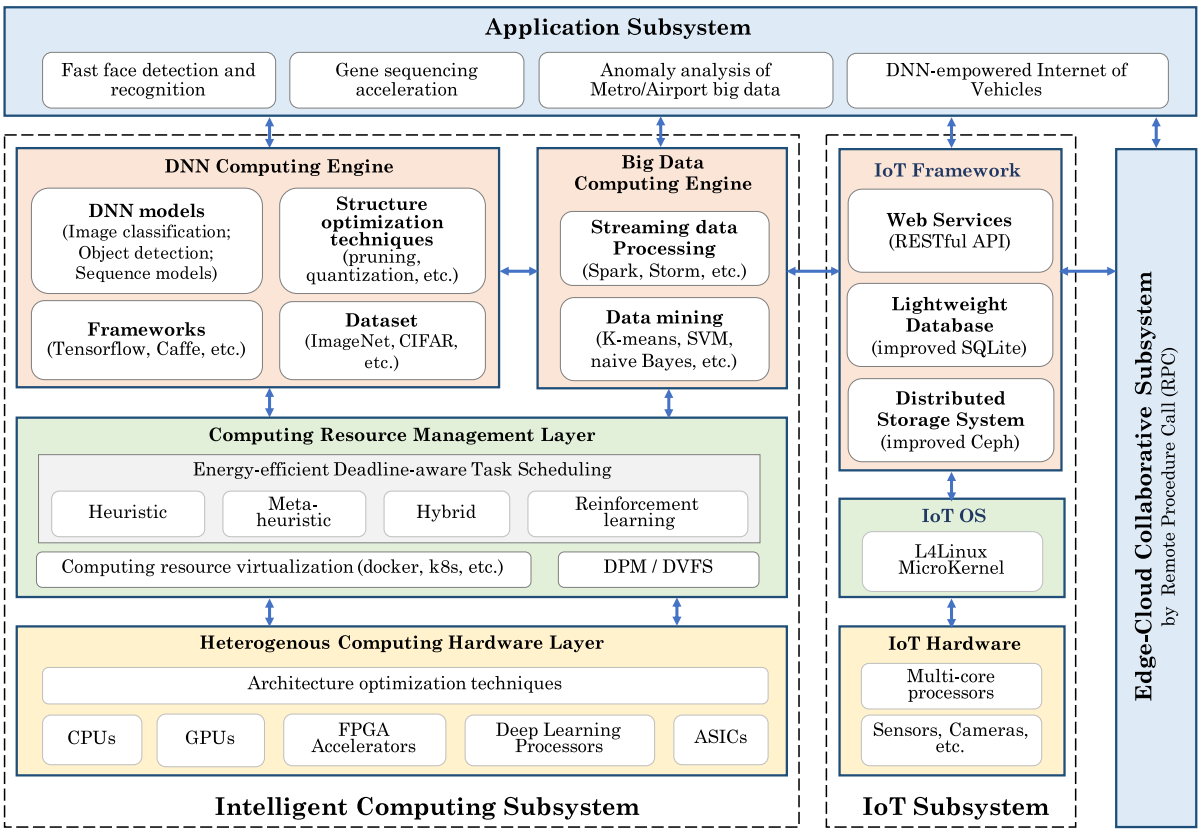

Fig. 2. System architecture of the AIoT-empowered ECCC system.

packages called containers. Docker containers are lightweight, and several containers can run simultaneously in a single server or virtual machine. Docker technology is suitable to be applied on the edge servers that are not particularly rich in resources, and it can be implemented by Kubernetes (k8s), which is an open-source container-orchestration system to automate the computer application deployment, scaling, and management.

3) Intelligent Computing Algorithm Layer: In the algorithm layer, the deep neural networks (DNNs) computing engines and big data computing engines are developed.

The DNN computing engines have implemented a set of DNN models, such as image classification models (AlexNet, VGG, GoogLeNet, ResNet, and so on), object detection models (R-CNN, YOLO, SSD, and so on), and sequence models (RNN, LSTM, GRU, and so on). In addition, the frameworks (Tensorflow, Caffe, and so on) and data set (ImageNet, CIFAR, and so on) dedicated to these models are also equipped. Furthermore, a serial of model structure optimization techniques, such as network pruning, network quantization, teacher–student network, and tensor decomposition, are also applied in these computing engines to optimize both the DNN computation time and energy cost.

The big data computing engines consist of the streaming data processing engines such as spark and storm and the data mining algorithms such as K-means, support vector machine (SVM), and naive Bayes. The deployment of the above computing engines enables the edge servers to process large amounts of data at high speed so that some computation-intensive tasks such as big data mining can be realized in real time.

4) IoT Subsystem: The IoT subsystem is composed of IoT hardware, IoT OS, and IoT framework.

IoT OS is improved based on L4Linux. L4 is the microkernel, which aims to provide security, isolation, and robustness for the IoT system. L4Linux runs as a service upon L4 microkernel and enables real-time programs to run on the edge servers in parallel at the same time.

The IoT framework consists of lightweight web service, lightweight database, and distributed storage system. The lightweight web service is based on the representational state transfer (REST) standards to improve the interoperability of web services. The lightweight database and storage system are improved on the basis of SQLite and Ceph, respectively. These mechanisms enable the data to be stored in a distributed and hierarchical way so that the query to the key data can be uninterrupted even if the backbone network is disconnected.

5) Edge-Cloud Collaborative Subsystem: The edge-cloud collaborative subsystem seeks to combine the advantages of high computing performance of the clouds and low latency of the edges. Task splitting, data compression, and remote procedure call (RPC) are the key technologies to achieve ECCC.

Take the ECCC of neural network (NN) as an example, and the task splitting technique is applied first to divide NN into edge partition and cloud partition from a specific layer. These two partitions are then performed by edge nodes

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

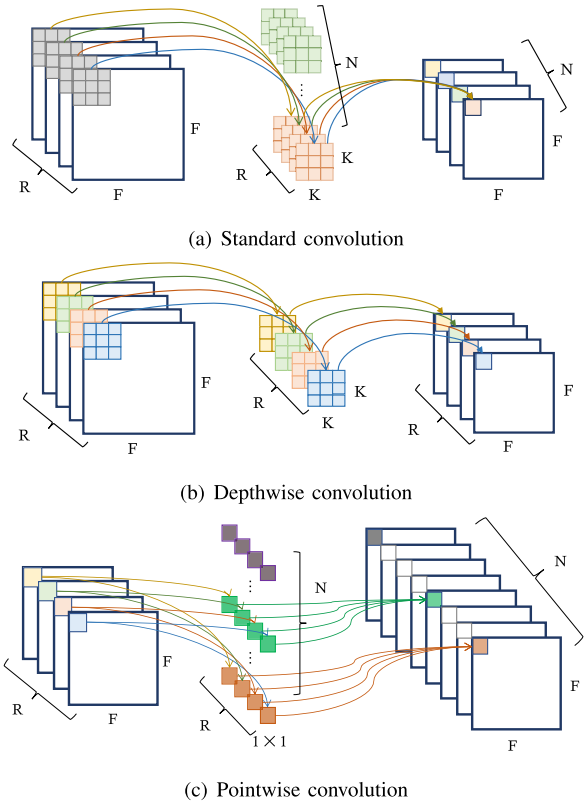

Fig. 3. Different convolution computations in MobileNet CNN. (a) SC. (b) Depthwise convolution. (c) Pointwise convolution.

and cloud nodes in a cooperative computing way. First, the edge node generates the output from the edge partition and then sends the output to the cloud node to make it continue the left computation based on the cloud partition. To reduce the communication load, the compression technique can be applied to the edges before sending data to clouds. Although compression technique will introduce compression and decompression overhead, it enables the transmission of smaller size packets and this may lead to overall better performance.

Based on the above work, the RPC technique can be further applied to realize collaborative computation between edges and clouds. RPC enables a procedure to be called on another computer on a shared network as if it was called locally. It does not require the developers to explicitly code the details for the remote interaction, and it enables the edge nodes to call remote programs on the clouds to make the subsequent work continue execution in the remote clouds.

6) Application Subsystem: The above AIoT-empowered ECCC system architecture can achieve joint optimization to energy consumption, computing latency, and algorithm accuracy. Currently, we have developed several emerging and cutting-edge applications based on this architecture, including the low-latency face recognition presented in this article, the fast gene sequencing, the real-time abnormal status monitoring for subway tracks, and the time-critical inference offloading for Internet of vehicles. In this section, we briefly introduce one application example that uses the above ECCC system to detect abnormal conditions in the subway tracks.

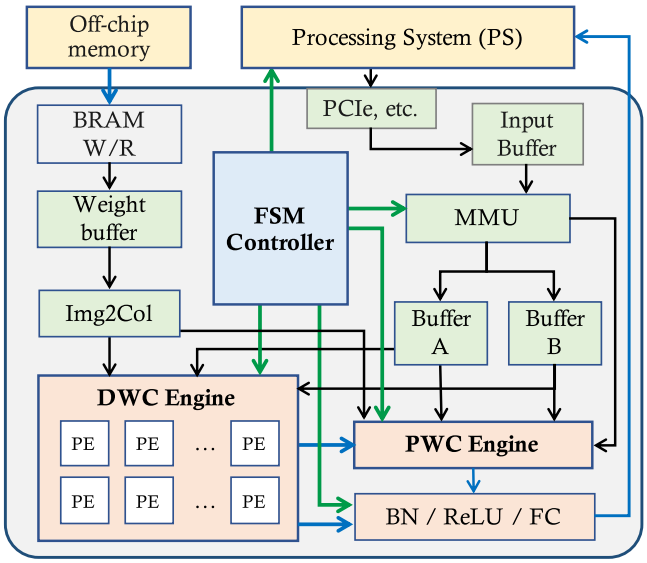

Fig. 4. Block diagram of FPGA-based MobileNet CNN accelerator.

Subway has played an important role in people's daily life. To guarantee the subway driving safety, it is necessary to assess the safety of subway tracks. In our work, we laid a lot of fiber optic sensors under the subway tracks to collect track status data. In case the tracks are deformed or broken, the transmission status of the optical signal in the optical fiber will change. By collecting data from the fiber optic sensors and performing real-time analysis to these data, the safety condition of the subway tracks can be assessed. However, the optical fiber sensors generate large amounts of data quickly, and it is hard to process these data in real time by the general-purpose CPUs. To meet this challenge, we classify the optical signals by clustering algorithm and then design an FPGA-based hardware accelerator dedicated to this algorithm. Later, we integrate this FPGA accelerator into the architecture presented in Fig. 2 so that a large-scale FPGA-based accelerating system can be built to process the subway big data in real time with low power.

IV. FPGA-Based Accelerator for MobileNet CNN

The above AIoT-empowered ECCC architecture has been applied in our work to implement a low-latency and energy-efficient FT system. The key technologies of this FT system include the DNN algorithm optimization, the DNN hardware accelerator design, and the heterogeneous task scheduling algorithm. Therefore, the following sections focus on presenting these three technologies. In this section, the DNN algorithm optimization and hardware accelerator design work are presented. In Section IV-A, the energy-efficient and time-aware task scheduling algorithm for the heterogeneous computing system is explored.

A. DSCs and MobileNet CNN

DSC is a form of factorized convolutions, which splits a standard convolution (SC) into two layers: a depthwise convolution (DWC) that applies a single filter for each input channel

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

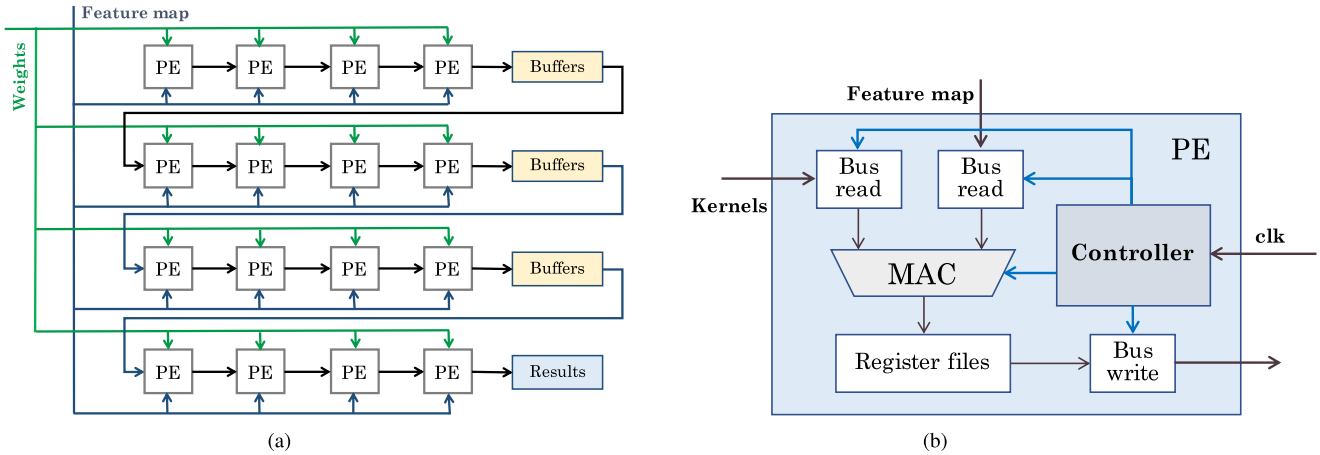

Fig. 5. Block diagram of depthwise convolution engine. (a) Depthwise convolution engine. (b) PE of depthwise convolution engine.

and a 1×1 pointwise convolution (PWC) that combines the output of the depthwise layer, as it is shown in Fig. 3. With DSC, the computing complexity and the model size of SC can be reduced drastically [14]. The above features make DSC appropriate to be applied in the edge servers, which require low latency, small memory footprint, and low power.

One typical instance that uses DSC is the MobileNet CNN. MobileNetV1 has 28 layers if counting the DWC and PWC as separate layers. The first convolution layer is the SC layer, whereas the followings are the DWC and PWC layers alternately. Except for the fully connected layer, all the other layers are followed by a rectified linear unit (ReLU) and batchnorm nonlinearity. MobileNetV2 improved MovileNetV1 by using an inverted residual structure between the thin bottleneck layers. It decreases the parameter size and improves accuracy further based on MobileNetV1.

B. General Block Diagram of FPGA-Based MobileNet CNN Accelerator

The general block diagram of the FPGA-based MobileNet CNN accelerator is shown in Fig. 4. It is composed of two parts: processing system (PS) and programmable logic (PL). The PL is in charge of running computation-intensive MobileNet CNNs specially, whereas PS is in charge of the other tasks, such as network control and data transmission. The parameters of MobileNet CNN models are stored on the off-chip memory, and they are loaded into the on-chip memory of PL during the run time. Two kinds of convolution engines are designed. One is in charge of performing DWC and SC, whereas the other is in charge of performing PWC. The double buffers A and B are used to provide efficient data flow for the computing engines, and the finite-state machine (FSM)-based controller is used to manage the operation procedures of the accelerator.

C. Architecture Design for MobileNet CNN Accelerator

This section presents the hardware architecture design for the FPGA-based MobileNet CNN accelerator. In addition, the CNN software optimization mechanism is also implemented to achieve hardware and software co-optimization for the CNN accelerator.

1) Computing Engine for Depthwise Convolution: The acceleration of DWC is realized from two aspects. One is the reduction of data access redundancy, and the other is the improvement of computing parallelism.

Data access redundancy exists because the moving stride of the kernel is commonly less than the width of the kernel matrix. Thus, when the kernel matrix moves through the feature map, some overlapped data are accessed repeatedly, which increases the energy cost and lowers the executing speed.

To reduce the data access redundancy and increase computing parallelism, a computing engine for accelerating DWC is designed, as shown in Fig. 5(a). A series of processing elements (PEs) build a systolic computing array. These PEs perform MAC operations currently to improve the convolutional computing speed. At the end of each row in this systolic array, a buffer is used to cache the data that will be reused in the near future. This buffer prevents these data from being discarded and avoids the redundant memory operations of reloading them from the off-chip memory.

Each PE is composed of an MAC, a controller, the register files, and the read/write buses, as shown in Fig. 5(b). The controller controls the loading of input data, as well as the transmission of output to the next PE. The kernel weights are stored in PEs only once and then keep still inside PEs. The elements of the feature maps flow into the PE arrays in a streaming way to enable the PEs to perform MAC computations.

When the data in all PEs are ready, these PEs run in parallel per cycle in the pipelining mode so that the executing speed of convolutional computation can be accelerated significantly. Since the feature map data only need to be loaded from off-chip memory to PEs for one time in this case, the data movement cost decreases significantly, thereby increasing the computing efficiency of the accelerator.

The ideal dimension of the PE array is the same as that of kernel matrix. However, due to the limitation of FPGA hardware resource, it may not be possible to build such a

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

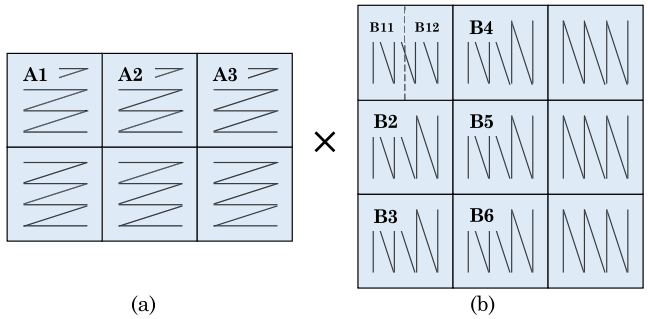

Fig. 6. Tiling technique for matrix multiplication. (a) FMM. (b) KWM.

large PE array inside the accelerator. Therefore, the matrix tiling technique is applied, which allows the data to be processed one by one in smaller blocks. As it is shown in Fig. 6, the feature map matrix (FMM) and kernel weight matrix (KWM) are divided into a set of smaller blocks, and then, the multiplication of FMM and KWM can be realized by performing matrix multiplication on these block matrices, e.g., multiply A1 by B1, A2 by B2, and A3 by B3, and then accumulate these multiplication results to work out the partial product of matrix multiplication of FMM and KWM. In this way, there is no necessity to design a large-scale PE array to process the entire matrix at one time. Consequently, the requirements for the hardware resources of FPGA-based accelerator are decreased significantly. Note that the tiling technique can be further applied to divide B1 into B11 and B12 so that the multiplication of A1 by B1 can be realized by multiplying A1 by B11 plus multiplying A1 by B12.

2) Computing Engine for Pointwise Convolution: As PWC has a different convolutional computing process from DWC, an independent computing engine is designed, especially for PWC to achieve high PWC computing efficiency.

The block diagram of the PWC computing engine is shown in Fig. 7(a). A set of PEs are equipped inside the computing engine. Each PE is in charge of performing convolutional computation between one kernel and the feature maps. Since the dimension of kernels in PWC is 1×1, the PWC computation can be regarded as the linear combination of the different feature maps. Therefore, the data from different feature matrices need to be extracted and reorganized before sending to PEs. This extraction and reorganization functionality is achieved by the memory management unit (MMU).

The structure of PEs in PWC computing engine is shown in Fig. 7(b). Kernels and feature maps are input into PEs to perform MAC operations inside PEs, and the output is stored into the buffer to be used as the input of the next DWC layer.

The same as DWC computing engine, the PWC computing engine may also not be able to process all the feature map and kernel data at one time due to hardware resource limitations. Therefore, the matrix tiling technique used in the DWC computing engine is also applied in the PWC computing engine.

3) Memory Management Unit: DWC and PWC have different ways to access the data in the feature maps. To satisfy the different data management requirements of these two computing engines, an MMU is designed inside the accelerator.

Suppose that feature map is a 3-D data space, and then, we can think that DWC extracts the feature map data from

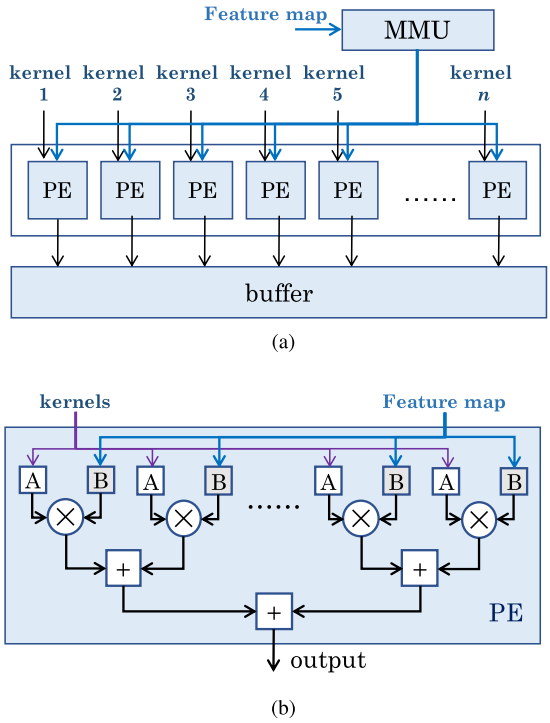

Fig. 7. Block diagram of pointwise convolution engine. (a) Pointwise convolution engine. (b) PE of Pointwise convolution engine.

the horizontal phase of this 3D space during the convolutional computing process, whereas PWC extracts the feature map data from the vertical phase of this 3-D space, as shown in Fig. 3. In other words, the DWC and PWC have different ways to extract the feature map data.

To provide efficient data access for these two computing engines, an MMU is implemented, as shown in Fig. 8. The feature map data are stored in a series of partitioned banks inside FPGA. In Fig. 8(a), the high-order part of address is used as the base address to perform feature map selection, whereas the low-order part of address is used to extract the data from this feature map. As the address increases or decreases, the data will be extracted from one feature map, that is, the data are accessed in the horizontal phase, and this case is appropriate for the DWC engine. In Fig. 8(b), inversely, the low-order part of address is used as the base address to select different feature maps, whereas the high-order part of address is increased or decreased to extract data from a given address in the feature map banks. In this case, the data are extracted in the vertical phase, and this way is appropriate for the PWC computing engine.

4) Memory System Design: The memory system of MobileNet CNN accelerator is described in Fig. 4.

The weight parameters are stored in off-chip memory and loaded into FPGA during the run time. Before the weight parameters are input to DWC and PWC computing engines, they need to perform an $img2col$ conversion operation. This operation plays an important role in converting the convolution computation into general matrix multiplication computation.

The input image data are transferred from PS to FPGA through the interfaces, such as PCIe or AXI. The input data

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

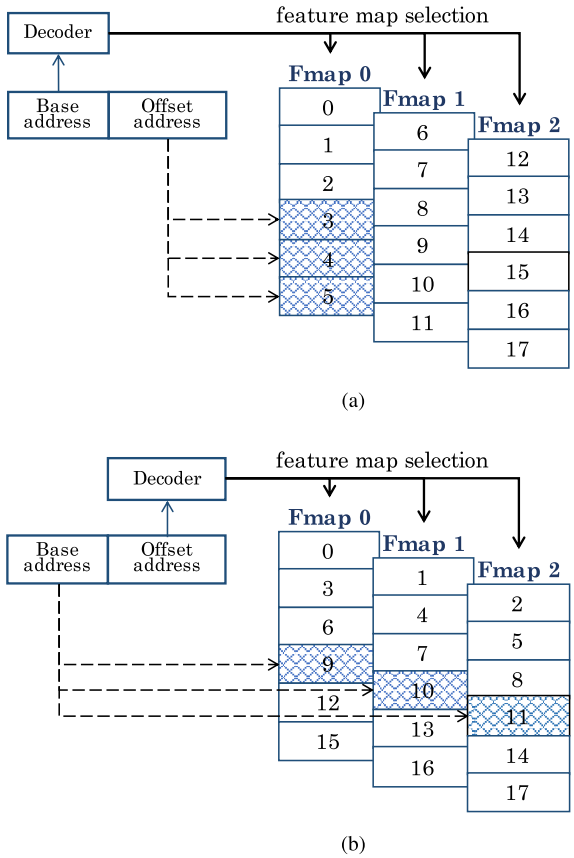

Fig. 8. Memory management mechanism. (a) Memory data prepared for DWC computation. (b) Memory data prepared for PWC computation.

need to be transformed by the MMU before inputting into DWC and PWC computing engines.

The ping-pong double buffer mechanism is implemented inside the accelerator to relieve the imbalance between computing speed and storage speed. Two buffers, which are buffers A and B, are built inside the accelerator. When buffer A transfers data to the computation engine, buffer B can load input data from PS simultaneously. Next, buffer B transfers data to computation engine; meanwhile, buffer A can load input data from PS. This double buffer scheme can ease the memory access bottleneck problem so that the computing efficiency of DWC and PWC can be improved significantly.

5) Other Optimization Techniques: This work also implements three types of computing parallelism proposed in [31] to accelerate the CNNs. In addition, the pipeline scheduling is implemented to maximize the throughput. As different pipeline stages have different execution times, more hardware resources are placed at the computation-intensive stages to keep the pipeline balanced.

The DNN software optimization approach, such as the data quantization, has also been implemented in the accelerator. It has been proved that it is effective to use the low-precision fixed-point arithmetic to substitute the floating-point arithmetic for the CNN training and inference [32], [33]. The work in [33] shows that the energy cost of 16-bit fixed-point multiplication is 6.2× less than the 32-bit floating-point multiplication, whereas the prediction accuracy loses less than 0.5%. Therefore, the low-precision fixed-point arithmetic, which uses 16-bit parameters and 13-bit input, is applied in our MobileNet CNN accelerator to not only decrease the hardware structure complexity but also improve the computing efficiency.

V. Task Scheduling for Heterogeneous System

As presented in Section III, a CPU/GPU/FPGA heterogeneous computing system based on FPGA accelerators is built to enable the hardware to adapt flexibly to the various software run-time contexts. Once this heterogeneous computing architecture is constructed, it is essential to explore a delay-aware and energy-efficient task scheduling algorithm dedicated to this architecture. In this section, we build the energy and time models of the heterogeneous system, formulate the scheduling problem, and then propose a scheduling algorithm to solve this problem.

A. Energy and Time Models of Heterogeneous System

1) Task Model: The application model of heterogeneous CPU/GPU/FPGA system can be represented by directed acyclic graph (DAG). Let $\boldsymbol{P} = \{p_1, p_2, \ldots, p_{|P|}\}$ denote the processor set, in which $|P|$ is the total processor number. Denote the DAG application model as $\boldsymbol{A} = \{\boldsymbol{N}, \boldsymbol{E}, \boldsymbol{T}\}$, where $\boldsymbol{N}$ represents the task set, $\boldsymbol{E}$ represents the communication cost between the nodes, and $\boldsymbol{T}$ represents the execution time. Suppose that n_i ($n_i \in \boldsymbol{N}$) represents a task, $e_{i,j}$ ($e_{i,j} \in \boldsymbol{E}$) represents the communication time from task n_i to task n_j and t_{ij} ($t_{ij} \in \boldsymbol{T}$) represents the execution time of running task n_i on the processor p_j with the maximum frequency.

2) Energy Model: Since the DVFS technique can strike the tradeoff between energy cost and execution time, it is very suitable to be applied in the problems that aim at minimizing energy consumption under time constraints. Therefore, we apply the DVFS technique to the heterogeneous CPU/GPU/FPGA system. In this case, the system power at frequency f can be defined as [34]

$$P(f) = P_s + r(P_i + P_d) = P_s + r\left(P_i + C \times f^n\right) \tag{1}$$

in which P_s, P_i, and P_d represent the static power, frequency-independent dynamic power, and frequency-dependent dynamic power, respectively. r represents the system status, which is equal to 1 when the system is active and 0 when the system is inactive. C and n represent the effective capacitance and dynamic power exponent, respectively.

As the system is heterogeneous and has DVFS support, the parameters P_s, P_i, P_d, C, n, and f in (1) are vectors. Define $P_{k,s}$, $P_{k,i}$, $P_{k,d}$, C_k, and n_k as the parameters on processor p_k, and $f_{k,l}$ as the l-level working frequency on processor p_k, and then, the energy cost of running task n_i on the processor p_k with the frequency $f_{k,l}$ can be denoted by

$$E\left(n_i, p_k, f_{k,l}\right) = P_{k,l} \times t_{i,k} \times \frac{f_{k,\max}}{f_{k,l}}. \tag{2}$$

The communication energy cost between tasks T_i and T_j can be defined as

$$e_{i,j} = P_{\text{comm}} \times \text{CL}_{i,j}/B_{i,j} \tag{3}$$

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

in which P_{comm}, $\text{CL}_{i,j}$, and $B_(i, j)$ represent the communication power, communication load, and communication bandwidth between tasks T_i and T_j, respectively.

Then, the energy cost of the heterogeneous system can be expressed as

$$E_{\text{sum}} = \sum_{i=1}^{|N|} E\left(n_i, p_k, f_{k,l}\right) + \sum_{i=1}^{|N|}\sum_{j=1}^{|N|} e_{i,j}. \tag{4}$$

3) Time Model: Suppose that the predecessor of task n_i is n_j. Then, the earliest starting time of task n_i in the heterogeneous system can be computed as

$$t(n_i)_{\text{start}} = \text{Fin}\left(n_j\right) + e_{i,j} \tag{5}$$

where $\text{Fin}(n_j)$ represents the completion time of task n_j.

Suppose that the time when the processor p_k is available to start is $T[k]$. Then, the actual start time of task n_i on the processor p_k can be computed by

$$\text{start}(n_i, p_k) = \max(t(n_i)_{\text{start}}, T[k]) \tag{6}$$

and the earliest finish time of the task n_i on the processor p_k with the working frequency $f_{k,l}$ can be computed as

$$\text{fin}\left(n_i, p_k, f_{k,l}\right) = \text{start}(n_i, p_k) + t_{i,k} \times \frac{f_{k,\max}}{f_{k,l}}. \tag{7}$$

B. Problem Formulation

The optimization objective of the scheduling algorithm is to minimize the task execution time under the energy constraint. Thus, the problem to be solved can be formulated as

$$\begin{cases} \min \quad \text{fin}(n_{\text{exit}}) \\ E_{\text{sum}} < E_{\text{limit}} \end{cases} \tag{8}$$

in which n_{exit} is the last task to be executed, $|N|$ represents the task number, and E_{limit} represents the energy constraint.

C. Selection of Scheduling Algorithm

Heuristic and metaheuristic algorithms are the two popular scheduling algorithms for heterogeneous environments. Heuristic algorithms are problem-dependent. They typically search for the optima in polynomial time by using incomplete information and show good performance in solving problems in a specific field, yet show poor performance in the others. Metaheuristic algorithms are problem independent. They usually search for the optimal solution in a random way and are mostly not deterministic. They are able to solve problems with high computational complexity and can be applied in various fields with acceptable performance. However, they have low search efficiency and high computational cost.

The heterogeneous earliest-finish-time (HEFT) [35], which is a well-known instance of list-based heuristic algorithms, has been popularly applied in many scheduling cases due to its low computational complexity. HEFT can reduce the searching space to a minimum range so that a higher searching efficiency can be achieved. Thus, in our work, we seek to develop a heterogeneous scheduling algorithm based on HEFT for the purpose of minimizing the makespan under the energy constraint of FT tasks. However, the HEFT algorithm has poor flexibility and performs well only for some specific application tasks. Consequently, it is appropriate only for the applications with small-scale tasks and is hard to satisfy the requirements of complex large-scale scheduling problems [36].

Algorithm 1 Algorithm for Q-Learning-Based HEFT

Input: $n \in \boldsymbol{N}$, $t \in \boldsymbol{T}$, $p \in \boldsymbol{P}$, f
Output: $Q[s, a]$

procedure $Initialization()$
  Initialize $Q(s, a)$ rabitrarily
  Initialize a state $s \leftarrow (n, p, f)$
  Initialize an action element $a \leftarrow (p, f)$
  Compute the initial energy cost E and time cost t
  return s, a
procedure $State_transfer(s, a, r, E, t)$
  Choose action $a \leftarrow Q(\gamma - greedy, \gamma \in (0, 1))$
  Update $s' \leftarrow= (i + 1) mod N$
  Compute rank_u and rank_d by traversing graph
  Compute $priority(n_i)$ in terms of equation 9
  Calculate E and t based on equation 4 and 7.
  $r \leftarrow 1$ if $E < E_{last}$ & $t < t_{limit}$
  If r equal to 1, $Q_{value} \leftarrow priority(n_i)$
  return s', r
procedure QL_update
  $s, a \leftarrow Initialization()$
  $episodes \leftarrow$ cycle counts
  for i between 0 to $episodes$ **do**
    $s', r \leftarrow State_transfer(s, a, r, E, t)$
    $a' \leftarrow a_i$, if a_i maximums $Q[s, a]$
    Update α and β in equation 9
    Update Q-table $Q[s, a]$ in terms of equation 11 by using Q_{value}
    Update states and actions: $s \leftarrow s'$,$a \leftarrow a'$
return $Q[s, a]$

In recent years, the Q-Learning algorithm, which is a value-based reinforcement learning algorithm, has attracted increasing attention in searching for the optimal solutions to the task scheduling problems. Q-learning can interact directly with the environment by a trial and error way without the need of knowing the environment model. Moreover, it can learn from the environment feedback rather than the labeled training set. These features make the Q-learning algorithm appropriate to solve many combinatorial optimization problems, such as task scheduling. Therefore, in this article, we combine the Q-learning algorithm with HEFT algorithm, named QL-HEFT algorithm, to make HEFT adapt to a variety of different applications and meanwhile keep high searching efficiency. Based on the QL-HEFT algorithm, the ECCC system assigns each FT application task to the most suitable CPU or GPU or FPGA accelerator for execution. In this way, the FT applications can be completed with less time and low energy cost.

D. Q-Learning-Based HEFT Scheduling Algorithm

The key concept of QL-HEFT is to use the Q-learning mechanism to sort the tasks in an optimal order and schedule

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RESOURCE USAGE OF MOBILENET V2 ACCELERATOR ON DIFFERENT FPGAS

Components	Intel Arria 10 GX 900			Intel Stratix 10 GX1100		
	ALM	RAM	DSP	ALM	RAM	DSP
Computation & Control	58732 (17.3%)	72 (2.9%)	1428 (94%)	89732 (20%)	112 (2%)	1926 (74.3%)
Feature map buffer	3 (0%)	1638 (67.6%)	0 (%)	8 (0%)	2689 (49%)	0 (0%)
Weight buffer	186 (0%)	566 (23.3%)	0 (%)	6 (0%)	926 (17%)	0 (0%)
Others	4978 (1.5%)	8 (0.3%)	0 (%)	7638 (1.7%)	20 (0.4%)	0 (0%)
Totals	63899 (18.8%)	2284 (94.3%)	1428 (94%)	97384 (21.7%)	3738 (68.4%)	1926 (74.3%)

each selected task to the best processor. This involves the improvement to both the task prioritizing phase and processor selection phase of the traditional HEFT.

In the task prioritizing phase, the priority of each task in the QL-HEFT algorithm is no more dependent only on the upward rank value rank_u but depends on both the upward rank rank_u and downward rank rank_d, which can be denoted by

$$\text{priority}(n_i) = \alpha \times \text{rank}_u + \beta \times \text{rank}_d \tag{9}$$

in which the parameters α and β are updated by the Q-learning algorithm in terms of the environment feedback, and the objective of this learning process is to obtain an optimal task order list.

In the processor selection phase, QL-HEFT does not select the processors for each task by only considering the minimization of tasks' finish time but considers both the optimization to the tasks' completion time and the constraint to the tasks' energy cost.

The algorithm of QL-HEFT is shown in Algorithm 1. The action of the QL-HEFT algorithm can be denoted as

$$A = \{(p, f, \alpha, \beta) | p \in \boldsymbol{P}, f \in \boldsymbol{F}\} \tag{10}$$

in which p represents the processor a task runs, f represents the working frequency the processor works at, and α and β represent the parameters to compute the task priority. The reward of the QL-HEFT algorithms depends on the evaluation of the time and energy consumption of the tasks, and it guides QL-HEFT how to update the Q value to search for a better solution.

During the learning process, the agent selects an action and moves to the new state. Then, QL-HEFT calculates the rewards and updates the Q-table by the formula as follows:

$$\begin{aligned} Q(s,a) \leftarrow\ & Q(s,a) + \alpha \\ & \times \left(r(s,a) + \gamma \times \max\left(Q'\left(s', a'\right)\right) - Q(s,a)\right). \end{aligned} \tag{11}$$

After a series of learning updates, the optimal task scheduling result can be generated and is shown in the converged Q-table.

VI. EXPERIMENTAL RESULTS

In this section, we conduct real-world experiments to evaluate the performance of the FPGA-based MobileNet CNN accelerators, the CPU/GPU/FPGA heterogeneous accelerating system, as well as the heterogeneous task scheduling algorithm.

A. Environment Setup

The experiment runs the facial image classification and facial object detection applications on the CPU, CPU/GPU, CPU/FPGA, and CPU/GPU/FPGA computing systems. The Intel Gold 5118 and Nivida Tesla P100 are used as the CPU and GPU platforms, respectively. As for FPGAs, both the Intel Arria 10 GX 900 and Intel Stratix 10 GX1100 are used to conduct the experiments. GPUs and FPGAs are connected to CPUs as the coprocessors via PCIe interfaces.

The MobileNet v2 algorithm [15] is used to perform image classification. The MobileNet-SSD algorithm, which substitutes the standard CNNs inside SSD with MobileNet, is used to perform object detection. The CUDA and Intel Quartus Prime 16 are used for the development on GPUs and FPGAs, respectively. The Tensorflow framework is used for the deep learning algorithm implementation.

The computing architecture of this experiment is shown in Fig. 9. Each edge computing node is equipped with one CPU, one GPU, and one FPGA. Different edge nodes are clustered and managed by kubernetes through containers. Two-level task scheduling mechanisms are implemented to allocate the tasks to the computing units. First, the kubernetes master splits the tasks and distributes them to different edge computing nodes. Then, the heterogeneous scheduling algorithm is further applied on each edge computing node to determine how to allocate the tasks among CPUs, GPUs, and FPGAs.

B. Performance Evaluation to FPGA-Based CNN Accelerator

To evaluate the performance of FPGA-based CNN accelerator, we implement a small core MobileNet v2 accelerator on the Intel Arria 10 GX 900 and a big core MobileNet v2 accelerator on the resource-abundant Intel Stratix 10 GX1100 FPGAs, respectively, and compare their speed and power efficiency with CPUs, GPUs, and the other state-of-the-art FPGA-based MobileNet CNN accelerators.

1) Resource Usage: In Table I, the resource usage of different modules in MobileNet v2 accelerator on FPGA is demonstrated. Due to the usage of hardware design techniques such as DSC, low-precision fixed-point arithmetic, and matrix tiling, the MobileNet CNN accelerator is feasible to be deployed on the middle-end Intel Arria 10 GX900 FPGA. Since Intel Stratix 10 GX1100 FPGA has richer hardware resources than Arria 10 GX 900 FPGA, a more powerful MobileNet accelerator is implemented by building more PEs

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

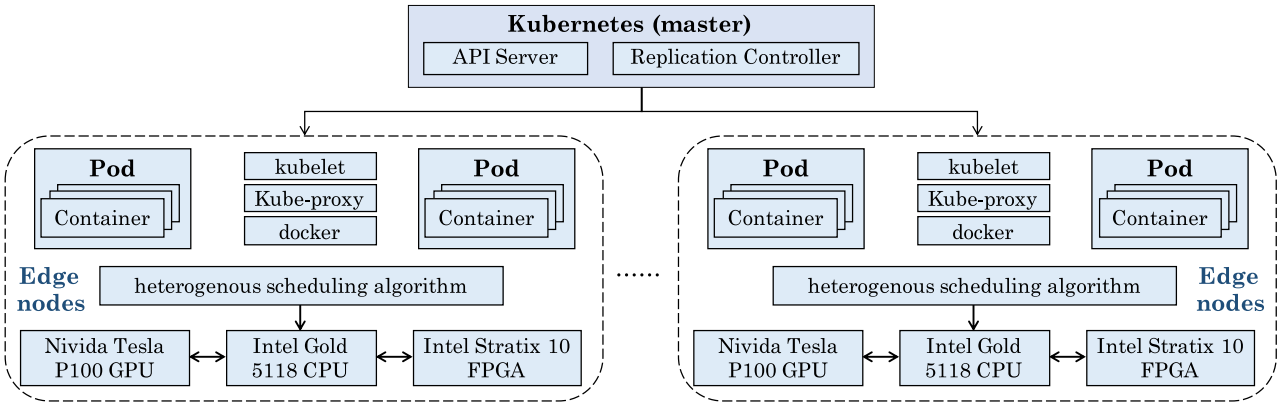

Fig. 9. Experimental platforms for edge nodes.

TABLE II
IMAGE CLASSIFICATION SPEED ON DIFFERENT FPGA ACCELERATORS

Design	**[15]**	**[38]**	**[39]**	**[40]**	**[41]**	**[41]**	**our work**	
Network	MobileNet v2	MobileNet v1	MobileNet v2	RR-MobileNet	MobileNet v2	MobileNet v2	MobileNet v2	
Platform	CPU	Stratix V	Arria 10	ZU9EG	ZU2EG	ZU9EG	Arria 10	Stratix 10
Speed (fps)	13.3	231.7	266.2	127.4	205.3	809.8	298	763.5

and using more DSPs to perform the computation. This causes more hardware resource consumption, yet results in better accelerating performance.

2) Speed Comparison With State-of-the-Art FPGA Accelerators: The computing speed of the FPGA accelerator in this article is compared with the state-of-the-art FPGA accelerators [15], [37]–[40] by running the facial image classification tasks, and the comparison result is shown in Table II.

The results show the MobileNet v2 accelerator on the Arria 10 FPGA in this article can achieve better performance than the state-of-the-art MobileNet accelerators, and there are several reasons for this. First, two independent computing engines are designed for DWC and PWC, respectively, which increases the computing performance. Second, an MMU dedicated to MobileNet is designed to improve the memory data organization efficiency. Third, large amounts of PEs are equipped in the computing engines to improve the computing parallelism, and the pipelining mechanism is also applied to increase the throughput. In addition, the data quantization technique has also reduced the computing complexity and meanwhile optimized the energy consumption.

3) Speed and Power Efficiency Comparison With CPUs and GPUs: We also evaluate the computing speed and power efficiency of the FPGA-based MobileNet CNN accelerator by running facial image classification applications on the CPUs, GPUs, and FPGAs.

The evaluation result is shown in Fig. 10. It shows that the MobileNet CNN accelerators based on Arria 10 and Stratix 10 FPGAs can achieve approximate or better computing performance than Nivida Telsa P100 GPU but have much lower energy consumption than GPU. Compared with CPU, FPGA achieves notable improvement in both computing speed and power efficiency. The above results show that FPGA accelerators can achieve good computing performance but still keep low energy cost. This is because the hardware architectures

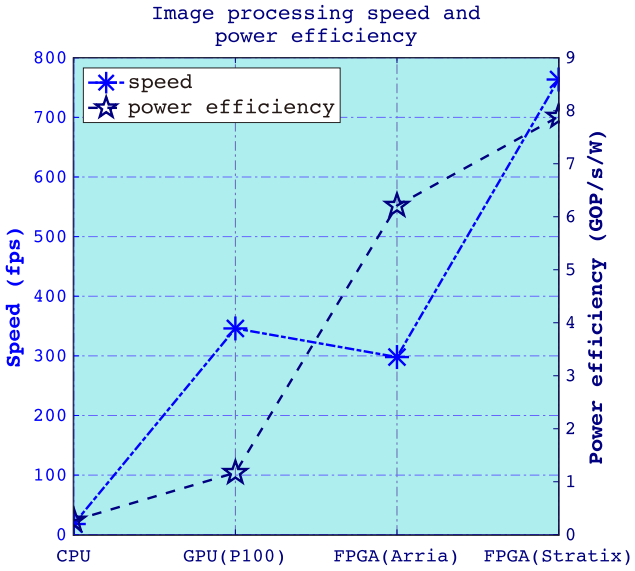

Fig. 10. Speed and power efficiency of image classification by comparing FPGAs with CPUs and GPUs.

of FPGA-based accelerators are designed specially in terms of the computing features and data access characteristics of CNNs, which makes the accelerators run CNNs with superior efficiency.

It is also shown from the results that the MobileNet CNN accelerator implemented on Intel Stratix 10 can have better power efficiency than that implemented on Arria 10. This is because more DSPs and block RAMs are directly used in Stratix 10 to perform the convolutional computation.

C. Performance Evaluation to CPU/GPU/FPGA Heterogeneous Acceleration Platform

We also conduct experiments to evaluate the makespan and convergence time of the QL-HEFT algorithm and also the

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

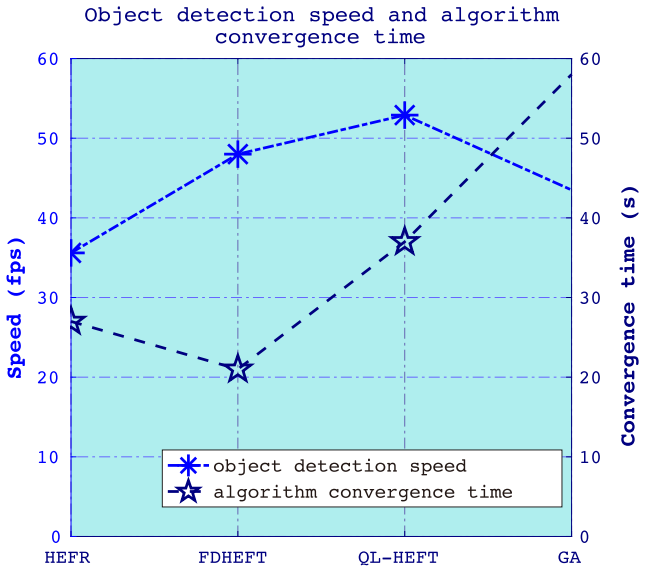

Fig. 11. Algorithm convergence time and object detection speed of ECCC systems using different algorithms.

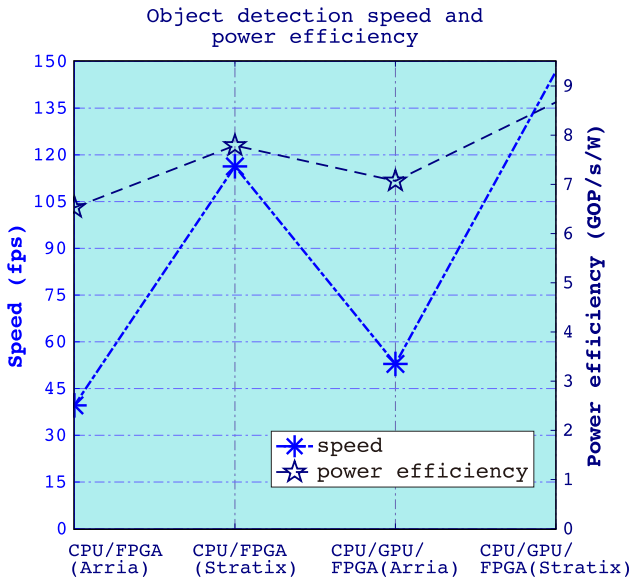

Fig. 12. Speed and power efficiency of object detection with CPU/FPGA and CPU/GPU/FPGA.

speed and power efficiency of CPU/GPU/FPGA heterogeneous computing system. As the smaller the makespan, the faster the processing speed. Therefore, we directly use the processing speed of face recognition to substitute makespan when evaluating the QL-HEFT algorithm.

1) Performance Evaluation to QL-HEFT Scheduling Algorithm: We run the MobileNet-SSD algorithm on CPU/GPU/FPGA (P100 GPU, Arria 10 FPGA) to perform face detection and use four scheduling algorithms to assign the tasks to CPUs, GPUs, or FPGAs. These algorithms are the traditional HEFT algorithm, the state-of-the-art FDHEFT [41], the metaheuristic genetic algorithm (GA), and the QL-HEFT algorithm proposed in this article.

We calculate the algorithm convergence time and object detection speed of the above four cases, and the result is shown in Fig. 11. The result shows that the system using the QL-HEFT algorithm can achieve better processing speed than the system using the others. This is because QL-HEFT has combined the advantages of HEFT's high searching efficiency and QL's strong learning ability. The result also shows that the QL-HEFT algorithm has shorter convergence time than GA. This is because QL-HEFT is based on the heuristic HEFT algorithm and has less searching complexity than the GA. Although the QL-HEFT algorithm has longer convergence time than HEFT and FDHEFT due to the iterative learning process, the increased time level is still acceptable. The above result proves that QL-HEFT is effective to be applied to schedule the tasks in the CPU/GPU/FPGA heterogeneous system.

2) Performance Evaluation to CPU/GPU/FPGA Computing System: We run the MobileNet-SSD algorithm to perform facial object detection on CPU/FPGA (Arria 10 FPGA), CPU/FPGA (Stratix 10 FPGA), CPU/GPU/FPGA (P100 GPU, Arria 10 FPGA), and CPU/GPU/FPGA (P100 GPU, Stratix 10 FPGA) computing platforms. The tasks of MobileNet-SSD are assigned to either CPUs, GPUs, or FPGA in terms of QL-HEFT scheduling algorithm, and the processing speed and power efficiency of these experimental cases are shown in Fig. 12.

The results show that the CPU/GPU/FPGA heterogeneous computing system can achieve higher computing performance and also better power efficiency compared with CPU/FPGA. There are several reasons for this: 1) different computation-intensive tasks in the CPU/GPU/FPGA computing system can be assigned to either GPU or FPGA in terms of the run-time requirements of each task, thereby improving the task execution efficiency, and (2) the GPU and FPGA in the CPU/GPU/FPGA computing system can work concurrently to improve the throughput. This result proves that the schemes proposed in this article are effective to decrease the latency and meanwhile keeping high energy efficiency of the FT system.

VII. Conclusion

This article proposes an AIoT-empowered ECCC system for the purpose of achieving joint optimization to not only the algorithm accuracy but also the energy consumption and computing latency. Based on this ECCC system architecture, a low-latency and energy-efficient FT system is developed.

To diminish the network congestion resulting from offloading all tasks to remote clouds, the ECCC mechanism is applied by transmitting only the low-quality facial images to remote cloud servers. By this means, the response time of the FT system can be optimized greatly. Considering that the standard CNNs have high computing complexity and nontrivial computation time, the DSC-based MobileNet CNN is applied and an FPGA-based CNN accelerator dedicated to this MobileNet CNN is developed. In this way, the executing speed of CNNs can be improved significantly, whereas the power efficiency can also keep efficient. Since the FPGA accelerator is a specific processor and cannot adapt to the various contexts of FT applications, it is further

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

integrated with GPU and CPU to improve the adaptability to the diverse application contexts. Furthermore, dedicated to the integrated CPU/GPU/FPGA heterogeneous platform, a delay-aware energy-efficient scheduling algorithm is also investigated. By the above hardware and software codesign, an energy-efficient and time-aware FT system can be implemented. Experiments are conducted to evaluate the performance of the MobileNet CNN accelerator, the heterogeneous scheduling algorithm, and the CPU/GPU/FPGA heterogeneous computing platform. The results prove the effectiveness of these proposed schemes.

REFERENCES

[1] Y. Qi, S. Zhang, F. Jiang, H. Zhou, D. Tao, and X. Li, "Siamese local and global networks for robust face tracking," *IEEE Trans. Image Process.*, vol. 29, pp. 9152–9164, 2020.

[2] S. Yang, P. Luo, C. C. Loy, and X. Tang, "Faceness-net: Face detection through deep facial part responses," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 8, pp. 1845–1859, Aug. 2018.

[3] U. Mahbub, S. Sarkar, and R. Chellappa, "Segment-based methods for facial attribute detection from partial faces," *IEEE Trans. Affect. Comput.*, vol. 11, no. 4, pp. 601–613, Oct. 2020.

[4] R. Ranjan, V. M. Patel, and R. Chellappa, "HyperFace: A deep multi-task learning framework for face detection, landmark localization, pose estimation, and gender recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 1, pp. 121–135, Jan. 2019.

[5] Z. Deng, X. Peng, Z. Li, and Y. Qiao, "Mutual component convolutional neural networks for heterogeneous face recognition," *IEEE Trans. Image Process.*, vol. 28, no. 6, pp. 3102–3114, Jun. 2019.

[6] C. Ding and D. Tao, "Trunk-branch ensemble convolutional neural networks for video-based face recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 1002–1014, Apr. 2018.

[7] Q. Wang and G. Guo, "LS-CNN: Characterizing local patches at multiple scales for face recognition," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 1640–1653, 2020.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[9] Z. Ning *et al.*, "Distributed and dynamic service placement in pervasive edge computing networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 6, pp. 1277–1292, Jun. 2021.

[10] Z. Ning *et al.*, "Intelligent edge computing in Internet of vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, early access, Jun. 5, 2020, doi: 10.1109/TITS.2020.2997832.

[11] J. Zhu, J. Hu, M. Zhang, Y. Chen, and S. Bi, "A fog computing model for implementing motion guide to visually impaired," *Simul. Model. Pract. Theory*, vol. 101, May 2020, Art. no. 102015.

[12] X. Wang, Z. Ning, S. Guo, and L. Wang, "Imitation learning enabled task scheduling for online vehicular edge computing," *IEEE Trans. Mobile Comput.*, early access, Jul. 28, 2020, doi: 10.1109/TMC.2020.3012509.

[13] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, Feb. 2020.

[14] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*. [Online]. Available: http://arxiv.org/abs/1704.04861

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.

[16] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1251–1258.

[17] X. Yin and X. Liu, "Multi-task convolutional neural network for pose-invariant face recognition," *IEEE Trans. Image Process.*, vol. 27, no. 2, pp. 964–975, Feb. 2018.

[18] S. N. D. Chua, S. F. Lim, S. N. Lai, and T. K. Chang, "Development of a child detection system with artificial intelligence using object detection method," *J. Electr. Eng. Technol.*, vol. 14, no. 6, pp. 2523–2529, Nov. 2019.

[19] J. Xu, "A deep learning approach to building an intelligent video surveillance system," *Multimedia Tools Appl.*, vol. 80, pp. 5495–5515, 2021, doi: 10.1007/s11042-020-09964-6.

[20] K. Cai, X. Miao, W. Wang, H. Pang, Y. Liu, and J. Song, "A modified YOLOv3 model for fish detection based on MobileNetv1 as backbone," *Aquacultural Eng.*, vol. 91, Nov. 2020, Art. no. 102117.

[21] L. Du *et al.*, "A reconfigurable streaming deep convolutional neural network accelerator for Internet of Things," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 1, pp. 198–208, Jan. 2018.

[22] P. Lei, J. Liang, Z. Guan, J. Wang, and T. Zheng, "Acceleration of FPGA based convolutional neural network for human activity classification using millimeter-wave radar," *IEEE Access*, vol. 7, pp. 88917–88926, 2019.

[23] W. Ding, Z. Huang, Z. Huang, L. Tian, H. Wang, and S. Feng, "Designing efficient accelerator of depthwise separable convolutional neural network on FPGA," *J. Syst. Archit.*, vol. 97, pp. 278–286, Aug. 2019.

[24] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[25] A. Aimar *et al.*, "NullHop: A flexible convolutional neural network accelerator based on sparse representations of feature maps," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 3, pp. 644–656, Mar. 2019.

[26] Y. Lu, C. Wang, L. Gong, and X. Zhou, "SparseNN: A performance-efficient accelerator for large-scale sparse neural networks," *Int. J. Parallel Program.*, vol. 46, no. 4, pp. 648–659, Aug. 2018.

[27] Q. Xiao, Y. Liang, L. Lu, S. Yan, and Y.-W. Tai, "Exploring heterogeneous algorithms for accelerating deep convolutional neural networks on FPGAs," in *Proc. 54th Annu. Design Autom. Conf.*, Jun. 2017, pp. 1–6.

[28] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "[DL] a survey of FPGA-based neural network inference accelerators," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 1, pp. 1–26, Apr. 2019.

[29] M. U. Torun, O. Yilmaz, and A. N. Akansu, "FPGA, GPU, and CPU implementations of Jacobi algorithm for eigenanalysis," *J. Parallel Distrib. Comput.*, vol. 96, pp. 172–180, Oct. 2016.

[30] R. Li, Y. Dou, and D. Zou, "Efficient parallel implementation of three-point *viterbi* decoding algorithm on CPU, GPU, and FPGA," *Concurrency Comput., Pract. Exper.*, vol. 26, no. 3, pp. 821–840, Mar. 2014.

[31] M. Motamedi, P. Gysel, V. Akella, and S. Ghiasi, "Design space exploration of FPGA-based deep convolutional neural networks," in *Proc. 21st Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2016, pp. 575–580.

[32] J. Wu, C. Leng, Y. Wang, Q. Hu, and J. Cheng, "Quantized convolutional neural networks for mobile devices," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 4820–4828.

[33] S. Han *et al.*, "EIE: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 243–254, 2016.

[34] B. Zhao, H. Aydin, and D. Zhu, "Shared recovery for energy efficiency and reliability enhancements in real-time applications with precedence constraints," *ACM Trans. Design Autom. Electron. Syst.*, vol. 18, no. 2, pp. 1–21, Mar. 2013.

[35] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[36] Y. Xu, K. Li, J. Hu, and K. Li, "A genetic algorithm for task scheduling on heterogeneous computing systems using multiple priority queues," *Inf. Sci.*, vol. 270, pp. 255–287, Jun. 2014.

[37] R. Zhao, X. Niu, and W. Luk, "Automatic optimising CNN with depthwise separable convolution on FPGA: (Abstact Only)," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2018, p. 285.

[38] L. Bai, Y. Zhao, and X. Huang, "A CNN accelerator on FPGA using depthwise separable convolution," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 65, no. 10, pp. 1415–1419, Oct. 2018.

[39] J. Su *et al.*, "Redundancy-reduced mobilenet acceleration on reconfigurable logic for imagenet classification," in *Proc. Int. Symp. Appl. Reconfigurable Comput.* Cham, Switzerland: Springer, 2018, pp. 16–28.

[40] D. Wu *et al.*, "A high-performance CNN processor based on FPGA for MobileNets," in *Proc. 29th Int. Conf. Field Program. Logic Appl. (FPL)*, Sep. 2019, pp. 136–143.

[41] X. Zhou, G. Zhang, J. Sun, J. Zhou, T. Wei, and S. Hu, "Minimizing cost and makespan for workflow scheduling in cloud using fuzzy dominance sort based HEFT," *Future Gener. Comput. Syst.*, vol. 93, pp. 278–289, Apr. 2019.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.

Xing Liu received the Ph.D. degree from the LIMOS Laboratory, University Blaise Pascal, Clermont-Ferrand, France, and the School of Electronic Information, Wuhan University, Wuhan, China, in 2014.

He is currently an Associate Professor with the School of Computer Science and Technology, Wuhan University of Technology, Wuhan. His research interests include the Internet of Things, artificial intelligence, and embedded systems.

Jianfeng Yang received the bachelor's, master's, and Ph.D. degrees from the School of Electronic Information, Wuhan University, Wuhan, China, in 1998, 2002, and 2009, respectively.

He was a Visiting Research Scholar with Intel Corporation (Shanghai) in 2012, and a Senior Visiting Scholar with Northwestern University, Evanston, IL, USA, from 2015 to 2016. He is currently an Associate Professor with Wuhan University, where he leads the Internet and IT Laboratory. His research interests include networking, edge computing, and high-reliability real-time wireless communication.

Chengming Zou received the Ph.D. degree from the Wuhan University of Technology, Wuhan, China, in 2003.

He was a Visiting Research Scholar with the University of York, York, U.K., in 2008. He is currently a Professor with the School of Computer Science and Technology, Wuhan University of Technology. His research interests include the Internet of Things, artificial intelligence, and embedded systems.

Qimei Chen (Member, IEEE) received the Ph.D. degree in communication engineering from Zhejiang University, Hangzhou, China, in 2017.

From 2015 to 2016, she was a Visiting Research Scholar with the School of Electrical and Computer Engineering, University of California at Davis, Davis, CA, USA. She is currently an Associate Researcher with the School of Electronic Information, Wuhan University, Wuhan, China. Her general research interests include long term evolution in unlicensed (LTEU), wireless local area network (WLAN), vehicular to everything (V2X), edge intelligence, and energy-efficient communication techniques in next-generation wireless communication systems.

Xin Yan received the Ph.D. degree from the Wuhan University of Technology, Wuhan, China, in 2006.

He was a Visiting Research Scholar with the Technische Universiteit Delft, Delft, The Netherlands, in 2008. He is currently an Associate Professor with the School of Computer Science and Technology, Wuhan University of Technology. His research interests include mobile computing, and wireless networking and smart grid communications.

Yuao Chen received the bachelor's degree from the School of Information Engineering, Wuhan University of Technology, Wuhan, China, in 2020. He is currently pursuing the master's degree with the School of Electronic Information, Wuhan University.

His research interests include networking and wireless communication.

Chenran Cai is currently pursuing the bachelor's degree with the School of Computer Science and Technology, Wuhan University of Technology, Wuhan, China.

His research interests include big data and artificial intelligence.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:30:51 UTC from IEEE Xplore. Restrictions apply.